NIST UNIVERSITY

CSE- BTech

Subject: Design And Analysis of Algorithm(CSE-200)

Semester:3rd Semester **Year:**2025

Title of the Project :

N-QUEEN Problem Solver and Visualiser - Implement Using Backtracking with Visualisation.

Student Name	Roll Number
Abhishek Samal	**202456006**
N Jayant Rao	**202456078**

Name of the Faculty: Prof. Rabindra Kumar Shial

Comment:

Mark obtained:

CONTENTS

1. **Introduction**

2. **Application of Project**

3. **Objective**

4. **List of Concepts Used**

5. **Algorithm/Flowchart**

6. **Code & Output / Result**

7. **Real Life Application**

8. **Conclusion**

INTRODUCTION

The N-Queen problem is a classic challenge in computer science and mathematics that involves placing N queens on an N×N chessboard such that no two queens threaten each other. As the board size increases, the number of possible arrangements grows rapidly, making manual solutions inefficient and error-prone. Exploring all placements without a systematic strategy becomes complex, especially for larger values of N.

The "**N-Queen Problem Solver and Visualizer"** provides an automated and structured approach to understanding and solving this problem using the backtracking algorithm. It efficiently explores the chessboard, checks safe positions, and backtracks when necessary to arrive at a valid configuration. The visualizer enhances comprehension by displaying the board and queen placements clearly, making the solving process intuitive and interactive.

This project applies algorithm design principles, recursion, and optimization techniques to build a reliable and educational tool. With its combination of algorithmic logic and graphical visualization, it serves as a valuable learning resource for students, educators, and anyone interested in exploring constraint satisfaction problems.

APPLICATION

1. **Understanding Backtracking & Algorithm Design:**
   The N-Queen problem is widely used as a teaching example for backtracking. This solver helps students visualize how algorithms explore possible solutions, detect conflicts, and backtrack when necessary. Such interactive understanding strengthens algorithmic thinking and problem-solving skills.
2. **Constraint Satisfaction Problem (CSP) Demonstration:**
   The problem represents a classical Constraint Satisfaction Problem where multiple conditions (row, column, and diagonal safety) must be satisfied simultaneously.
   The system shows how constraints can be efficiently processed, making it useful for courses on artificial intelligence and optimization.

3. **Optimization and Search Techniques Learning Tool:**
   By visualizing recursive calls, safe checks, and backtracking steps, learners can observe how search spaces are pruned.
   This understanding is valuable in fields like AI search strategies, game theory, and combinatorial optimization.
4. **Graphical Visualization for Better Concept Clarity:**
   The visualization component helps users see how queens are placed step-by-step on the board.
   This is especially beneficial for students who struggle with abstract algorithmic concepts, as the board representation makes the logic more intuitive and engaging.
5. **Foundation for Advanced Algorithms & Real-World Solutions:**
   The N-Queen problem forms the basis for exploring more complex problems such as scheduling, resource allocation, and robotics path planning. Learning from this solver equips students with skills that apply to real-world algorithmic challenges, including AI planning and advanced CSP solvers.

OBJECTIVE

- To develop an algorithmic solution that places N queens on an N×N chessboard without conflicts.
- To ensure no two queens attack each other by preventing clashes along rows, columns, and diagonals.
- To apply backtracking as an efficient strategy for exploring valid queen placements.
- To analyze the complexity and performance of recursive search techniques used in constraint-based problems.
- To improve logical thinking by solving a classical combinatorial optimization challenge.
- To design a generalized solution that works for any value of N, not just the standard 8-queen version.
- To visualize the final arrangement of queens for better understanding and verification.

LIST OF CONCEPTS USED

- **Backtracking Algorithm:**

A systematic method of exploring all possible configurations by placing queens one by one. If a placement leads to a conflict, the algorithm "backtracks" and tries a new position.

- **Recursion:**

The solution relies heavily on recursive function calls to try placing queens row by row until a complete solution is found.

- **Constraint Checking (Row, Column, Diagonal Safety):**

Before placing a queen, the algorithm checks if the position is safe — ensuring no two queens attack each other.

- **2D Array / Chessboard Representation:**

The N×N board is represented using a 2D array or list of lists, where queens are placed and checked.

- **Time Complexity & Space Complexity Analysis:**

The N-Queen problem is computationally intensive. Its complexity is studied to understand how performance changes as N increases.

- **Output Visualization:**

Displaying the final board configuration using characters ('Q' for queen and '.' or '0' for empty spaces).

- **Generalization for Any N:**

The solution is designed to work not only for 8 queens but for any user-defined N.

ALGORITHM:

1. Start
2. Input the value of N (size of chessboard)
3. Initialize an N × N board with all positions empty
4. Start placing a queen in the first row
5. For each column in the current row:

Check if placing a queen here is safe (no attack from any other queen).

6. **If safe:**
   Place the queen
   Move to the next row (recursive step)
7. **If not safe:**
   Try the next column
8. **If all columns are tried and none are safe:**
   Backtrack → remove previous queen and try a new position
9. **If queens are placed in all N rows:**
   Solution found; display or store the board
10. Stop when all possible solutions are explored
11. End

Flowchart (Text Format):

```
        START
          |
      [Input N]
          |
[Initialize N×N Board]
          |
[Place Queen in Row 0]
          |
[Is Position Safe?]
      /       \
   YES        NO
     |         |
[Place]   [Try Next Column]
                |
      [Move to Next Row]
                |
      [All Rows Filled?]
            /       \
         YES        NO
            |         |
      [Print Solution]
                |
      [Backtrack if Needed]
                |
             END
```

CODE(In Java)

```
// NQueenSolver File:
public class NQueenSolver {
    private int N;
    public int[][] board;
    public NQueenSolver(int N) {
        this.N = N;
        board = new int[N][N];
    }

    // Function to check if placing queen is safe
    private boolean isSafe(int row, int col) {

        // Check row on left
        for (int i = 0; i < col; i++)
            if (board[row][i] == 1)
                return false;

        // Check upper diagonal
        for (int i = row, j = col; i >= 0 && j >= 0; i--, j--)
            if (board[i][j] == 1)
                return false;

        // Check lower diagonal
        for (int i = row, j = col; i < N && j >= 0; i++, j--)
            if (board[i][j] == 1)
                return false;

        return true;
    }

    // Backtracking recursive function
    public boolean solveNQueen(int col) {
        if (col >= N)
            return true;
for (int i = 0; i < N; i++) {
        if (isSafe(i, col)) {
                board[i][col] = 1;
```

```
if (solveNQueen(col + 1))
            return true;
            board[i][col] = 0; // Backtrack
        }
    }
    return false;
  }
}
    }
  }
  return 0;
}
```

// NQueenVisualizer File:

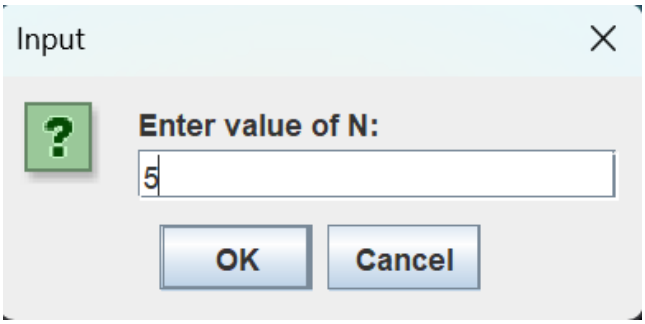
```
 import javax.swing.*;
import java.awt.*;
public class NQueenVisualizer extends JPanel {

   private int[][] board;
   private int N;

   public NQueenVisualizer(int[][] board, int N) {
      this.board = board;
      this.N = N;
      setPreferredSize(new Dimension(600, 600));
   }
   @Override
   protected void paintComponent(Graphics g) {
      super.paintComponent(g);

 int cellSize = 600 / N;

      for (int i = 0; i < N; i++) {
         for (int j = 0; j < N; j++) {
            // Chessboard colors
            if ((i + j) % 2 == 0)
               g.setColor(new Color(240, 217, 181)); // Light
            else
```

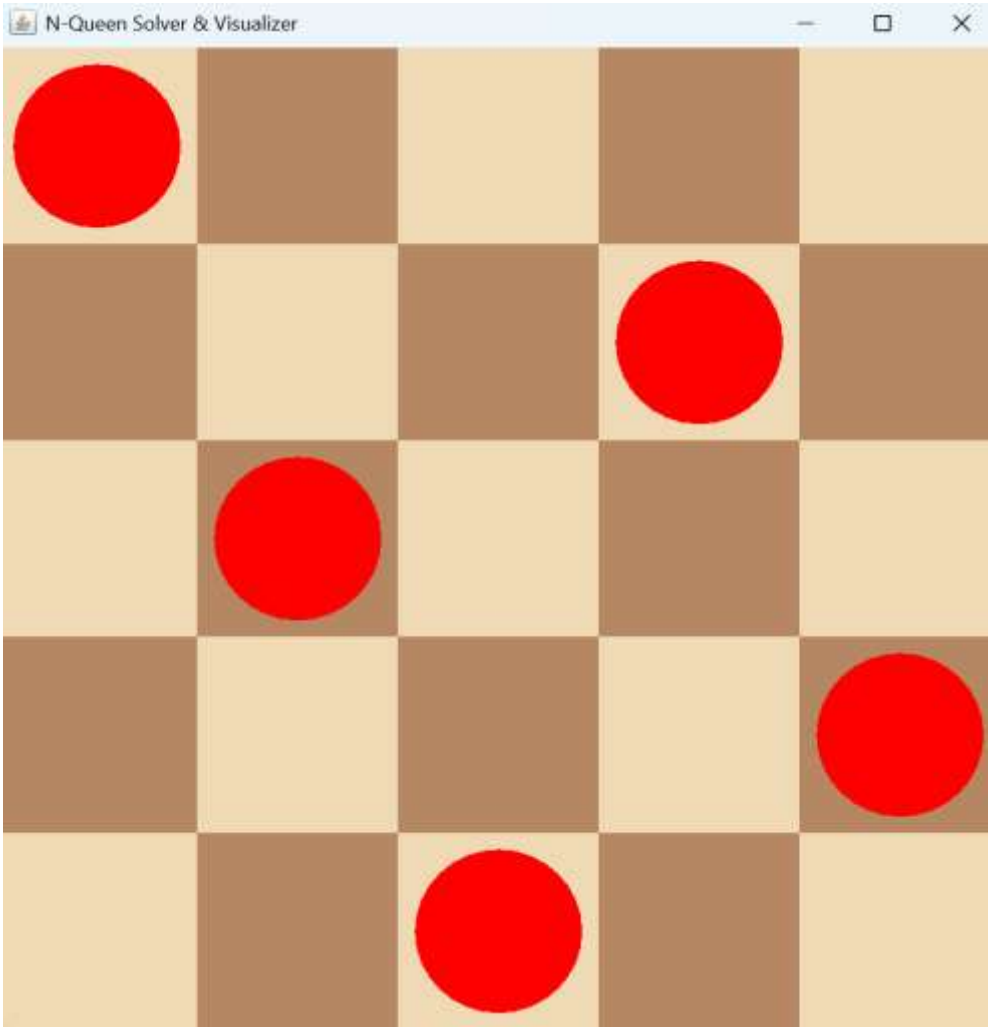
```
g.setColor(new Color(181, 136, 99));  // Dark

        g.fillRect(j * cellSize, i * cellSize, cellSize, cellSize);

            // Draw Queen
            if (board[i][j] == 1) {
                g.setColor(Color.RED);
                g.fillOval(j * cellSize + 10, i * cellSize + 10,
                        cellSize - 20, cellSize - 20);
            }
        }
    }
  }
}
```

// Main File:

```
import javax.swing.*;
public class Main {
   public static void main(String[] args) {
      String input = JOptionPane.showInputDialog("Enter value of N:");
      int N = Integer.parseInt(input);
      NQueenSolver solver = new NQueenSolver(N);
      if (solver.solveNQueen(0)) {
         JFrame frame = new JFrame("N-Queen Solver & Visualizer");
         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

         NQueenVisualizer visualizer = new NQueenVisualizer(solver.board, N);
         frame.add(visualizer);

         frame.pack();
         frame.setLocationRelativeTo(null);
         frame.setVisible(true);

      } else {
         JOptionPane.showMessageDialog(null,
               "No solution exists for N = " + N);
      }
   }
}
```

OUTPUT





Real-Life Applications of the N-Queen Problem (Backtracking):

1.CPU Task Scheduling (Avoiding Conflicts)

In multi-core processors, tasks must be assigned to CPU cores without causing conflicts. Just like queens cannot attack each other, tasks must be placed so they do not overlap or block each other.

N-Queen ↔ Scheduling tasks without collision

2. Wireless Communication (Frequency Assignment)

Telecommunication towers must use frequencies so that no two nearby towers interfere.

N-Queen ↔ Interference-free frequency allocation

3. Parallel Processing and Resource Allocation

In distributed systems, resources must be allocated to processes without clashes.

Example:

Assigning multiple jobs to multiple machines where no two jobs should use the same resources at the same time.

N-Queen ↔ Safe resource allocation

4. School Timetabling & Exam Hall Scheduling

Timetables require:

- No two classes with same teacher at same time
- No student writing two exams at same time
- Proper room allocation without overlap

This is similar to placing queens without conflict.

5. Sudoku Solvers (Same Backtracking Logic)

Although Sudoku is different, it uses the **same backtracking algorithm** as N-Queen.

- Try placing a number
- If conflict → backtrack
- Continue until solved

Great example to show real-world dependency of backtracking.

6. Game AI (Chess, Puzzles, Strategy Games)

Game engines use backtracking to:

- make moves
- undo invalid moves
- explore possible positions

The N-Queen is a simplified version of these state-space explorations.

CONCLUSION

The N-Queen Problem Solver and Visualizer developed in this project effectively demonstrates the power of backtracking algorithms in solving complex combinatorial problems. By systematically exploring possible queen placements and intelligently backtracking when conflicts occur, the program finds a valid configuration for any given value of N. The visualization component enhances understanding by showing how queens are arranged on the chessboard and how the algorithm arrives at the final solution.

This project bridges the gap between theoretical algorithmic concepts and practical implementation by showcasing how recursive backtracking operates behind the scenes. It also emphasizes the importance of algorithm efficiency, recursion, constraint checking, and problem decomposition—core principles of the Design and Analysis of Algorithms. Through this system, learners gain deeper insight into how real-world solvers handle constraint satisfaction problems such as scheduling, resource allocation, puzzle solving, and AI search strategies.

Overall, the N-Queen Solver and Visualizer successfully illustrates how algorithmic thinking can be transformed into an interactive, educational, and visually intuitive application. It highlights the relevance of backtracking in modern computing and strengthens foundational understanding of algorithm design and optimization.

REFERENCES

1. **Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C.** (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
2. **Levitin, A.** (2012). *Introduction to the Design & Analysis of Algorithms* (3rd ed.). Addison-Wesley.
3. **Sahni, S.** (2005). *Data Structures, Algorithms and Applications in Java* (2nd ed.). Universities Press.
4. **Weiss, M. A.** (2012). *Data Structures and Algorithm Analysis in Java* (3rd ed.). Pearson.
5. **Baase, S., & Van Gelder, A.** (2000). *Computer Algorithms: Introduction to Design and Analysis* (3rd ed.). Addison-Wesley.
6. GeeksforGeeks. (2024). *N-Queen Problem – Backtracking*. Retrieved from **https://www.geeksforgeeks.org/n-queen-problem-backtracking-3/**
7. W3Schools. (2024). *Java Programming Tutorial*. Retrieved from **https://www.w3schools.com/java/**